Ambientes virtuais
Referência:
https://docs.python.org/pt-br/3.15/tutorial/venv.html

12. Ambientes virtuais e pacotes

12.1. Introdução

Aplicações em Python normalmente usam pacotes e módulos que não vêm como parte da instalação padrão. Aplicações às vezes necessitam uma versão específica de uma biblioteca, porque ela requer que algum problema em particular tenha sido consertado ou foi escrita utilizando-se de uma versão obsoleta da interface da biblioteca.
Isso significa que talvez não seja possível que uma instalação Python preencha os requisitos de qualquer aplicação. Se uma aplicação A necessita a versão 1.0 de um módulo particular mas a aplicação B necessita a versão 2.0, os requisitos entrarão em conflito e instalar qualquer uma das duas versões 1.0 ou 2.0 fará com que uma das aplicações não consiga executar.
A solução para este problema é criar um ambiente virtual, uma árvore de diretórios que contém uma instalação Python para uma versão particular do Python, além de uma série de pacotes adicionais.
Diferentes aplicações podem então usar diferentes ambientes virtuais. Para resolver o exemplo anterior de requisitos conflitantes, a aplicação A deve ter seu próprio ambiente virtual com a versão 1.0 instalada enquanto a aplicação B vai possuir outro ambiente virtual com a versão 2.0. Se a aplicação B precisar fazer uma atualização para a versão 3.0, isso não afetará o ambiente da aplicação A.

12.2. Criando ambientes virtuais

O módulo usado para criar e gerenciar ambientes virtuais é chamado venv. venv instalará a versão do Python a partir da qual o comando foi executado (conforme relatado pela opção --version). Por exemplo, executar o comando com `python3.12` instalará a versão 3.12.
Para criar um ambiente virtual, escolha um diretório onde deseja colocá-lo e execute o módulo venv como um script com o caminho do diretório:

```
python -m venv tutorial-env
```

Isso irá criar o diretório `tutorial-env` se ele não existir, e também criará diretórios dentro dele contendo uma cópia do interpretador Python, a biblioteca padrão e diversos arquivos de suporte.
Um diretório de localização comum para um ambiente virtual é `.venv`. Esse nome tipicamente mantém o diretório oculto em seu ambiente, portanto é transparente, ao menos tempo que explica o motivo desse diretório existir. Também previne conflitos com `.env`, arquivos de definição de variáveis de ambiente que algumas ferramentas utilizam.
Uma vez criado seu ambiente virtual, você deve ativá-lo.
No Windows, execute:

```
tutorial-env\Scripts\activate
```

No Unix ou no MacOS, executa:

```
source tutorial-env/bin/activate
```

(Este script é escrito para o shell bash. Se você usa shells **csh** ou **fish**, existem scripts alternativos `activate.csh` e `activate.fish` para utilização.)

Ao ativar seu ambiente virtual haverá uma mudança no prompt do shell para mostrar qual ambiente virtual você está usando e modificará o ambiente para que quando você executar `python` ativar a versão e instalação do Python particular àquele ambiente. Por exemplo:

```
source ~/envs/tutorial-env/bin/activate
 python
Python 3.5.1 (default, May  6 2016, 10:59:36)
  ...
>>> import sys
>>> sys.path
['', '/usr/local/lib/python35.zip', ...,
'~/envs/tutorial-env/lib/python3.5/site-packages']
>>>
```

Note que o ambiente virtual ativado não altera `PYTHONPATH` variáveis de forma alguma. Isso pode levar a resultados inesperados se o path incluir referencia a código incompatível com a versão Python do ambiente virtual em uso. A melhor prática é usar `unset PYTHONPATH` no bash ou no shell que você está usando.
Para desativar um ambiente virtual, digite:

```
deactivate
```

no terminal.

12.3. Gerenciando pacotes com o pip

Você pode instalar, atualizar e remover pacotes usando um programa chamado **pip**. Por padrão `pip` irá instalar pacotes do Python Package Index. Você pode navegar pelo Python Package Index através do seu navegador web.
`pip` tem uma série de subcomandos: “install”, “uninstall”, “freeze”, etc. (Consulte o guia Instalando módulos Python para a documentação completa do `pip`.)
Você pode instalar a última versão de um pacote apenas especificando nome do pacote:

```
python -m pip install novas
Collecting novas
  Downloading novas-3.1.1.3.tar.gz (136kB)
Installing collected packages: novas
  Running setup.py install for novas
Successfully installed novas-3.1.1.3
```

Você também pode instalar uma versão específica de um pacote dando o nome do pacote seguido por `==` e o número da versão:

```
python -m pip install requests==2.6.0
Collecting requests==2.6.0
  Using cached requests-2.6.0-py2.py3-none-any.whl
Installing collected packages: requests
Successfully installed requests-2.6.0
```

Se você executar novamente o comando, `pip` será notificado de que a versão já está instalada, e não fará nada. Você pode fornecer um número de versão diferente para obter essa versão ou pode executar `python -m pip install --upgrade` para atualizar o pacote para a versão mais recente:

```
python -m pip install --upgrade requests
Collecting requests
Installing collected packages: requests
  Found existing installation: requests 2.6.0
    Uninstalling requests-2.6.0:
      Successfully uninstalled requests-2.6.0
Successfully installed requests-2.7.0
```

`python -m pip uninstall` seguido por um ou mais nomes de pacote removerá os pacotes do ambiente virtual. `python -m pip show` exibirá informações sobre um pacote específico:

```
python -m pip show requests
---
Metadata-Version: 2.0
Name: requests
Version: 2.7.0
Summary: Python HTTP for Humans.
Home-page: http://python-requests.org
Author: Kenneth Reitz
Author-email: me@kennethreitz.com
License: Apache 2.0
Location: /Users/akuchling/envs/tutorial-env/lib/python3.4/site-packages
Requires:
```

`python -m pip list` exibirá todos os pacotes instalados no ambiente virtual:

```
python -m pip list
novas (3.1.1.3)
numpy (1.9.2)
pip (7.0.3)
requests (2.7.0)
setuptools (16.0)
```

`python -m pip freeze` produzirá uma lista semelhante dos pacotes instalados, mas a saída usa o formato que `python -m pip install` espera. Uma convenção comum é colocar esta lista em um arquivo `requirements.txt`:

```
python -m pip freeze > requirements.txt
 cat requirements.txt
novas==3.1.1.3
numpy==1.9.2
requests==2.7.0
```

O arquivo `requirements.txt` pode ser submetido no controle de versão e adicionado como parte da aplicação. Usuários poderão então instalar todos os pacotes necessários com um `install -r`:

```
python -m pip install -r requirements.txt
Collecting novas==3.1.1.3 (from -r requirements.txt (line 1))
  ...
Collecting numpy==1.9.2 (from -r requirements.txt (line 2))
  ...
Collecting requests==2.7.0 (from -r requirements.txt (line 3))
```

```
  ...
Installing collected packages: novas, numpy, requests
  Running setup.py install for novas
Successfully installed novas-3.1.1.3 numpy-1.9.2 requests-2.7.0
```

pip tem inúmeras outras opções. Consulte o guia Instalando módulos Python para a documentação completa do pip. Quando você escrever um pacote e desejar deixá-lo disponível no Python Package Index, consulte o guia de usuário para empacotamento de Python.